4 - LightGBM

1. Comparison between $LightGBM$ and $XGBoost$

$LightGBM$ 可以看成是 $XGBoost$ 的升级加强版本

正如其名字中的 $Light$ 所蕴含的那样，和 $XGBoost$ 相比，$LightGBM$ 在大规模数据集上跑起来更加轻盈。

- 模型精度：$XGBoost$ 和 $LightGBM$ 相当。
- 训练速度：$LightGBM$ 远快于 $XGBoost$。
- 内存消耗：$LightGBM$ 远小于 $XGBoost$。
- 缺失值特征：$XGBoost$ 和 $LightGBM$ 都可以自动处理特征缺失值。
- 分类特征：$XGBoost$ 不支持类别特征，需要 $OneHot$ 编码预处理。$LightGBM$ 直接支持类别特征。

2. Performance Optimization Principle of LightGBM

$LightGBM$ 在 $XGBoost$ 上主要有 3 方面的优化。

1. $Histogram$ 算法: 直方图加速算法。
2. $GOSS$ 算法: 基于梯度的单边采样算法。
3. EFB 算法: 互斥特征捆绑算法。

可以用如下一个简单公式来说明 $LightGBM$ 和 $XGBoost$ 的关系：

$$LightGBM = XGBoost + Histogram + GOSS + EFB$$

那么，$Histogram$ 算法，$GOSS$ 算法，和 EFB 算法分别从什么角度对 $XGBoost$ 进行性能优化呢？

我们先概括性地从全局进行分析，然后再逐个加以介绍。

$XGBoost$ 模型训练的总体的复杂度可以粗略估计为：

$$\text{训练复杂度} = \text{树的棵数} \times \text{每棵树上叶子的数量} \times \text{生成每片叶子的复杂度}$$

由于 $XGBoost$ 采用的基模型是二叉树，因此生成每片叶子需要分裂一次。

而每次分裂，需要遍历所有特征上所有候选分裂点位，计算按照这些候选分裂点位分裂后的全部样本的目标函数增益，找到最大的那个增益对应的特征和候选分裂点位，从而生成一片新叶子。因此生成一片叶子的复杂度可以粗略估计为：

$$\text{生成一片叶子的复杂度} = \text{特征数量} \times \text{候选分裂点位数量} \times \text{样本的数量}$$

而 $Hitogram$ 算法的作用是减少候选分裂点位数量，$GOSS$ 算法的作用是减少样本的数量，EFB 算法的作用是减少特征的数量。

通过这 3 个算法的引入，$LightGBM$ 生成一片叶子需要的复杂度大大降低了，从而极大节约了计算时间。

同时 $Histogram$ 算法还将特征由浮点数转换成 $0 \sim 255$ 位的整数进行存储，从而极大节约了内存存储。

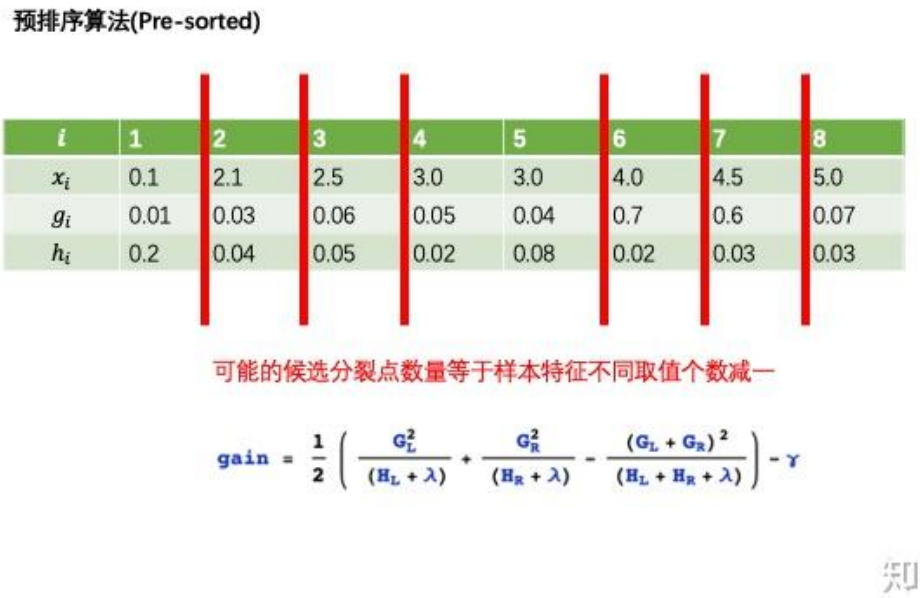
2.1 Histogram Algorithm

直方图算法是替代 $XGBoost$ 的预排序 $(pre-sorted)$ 算法的。

预排序算法首先将样本按照特征取值排序，然后从全部特征取值中找到最优的分裂点位，该算法的候选分裂点数量与样本数量成正比。

而直方图算法通过**将连续特征值离散化**到固定数量 (如 255 个) 的 $bins$ 上，使得候选分位点位为常数个 $(num_bins - 1)$。

此外，直方图算法还能够作**直方图差加速**。当节点分裂成两个时，右边叶子节点的直方图等于其父节点的直方图减去左边叶子节点的直方图。从而大大减少构建直方图的计算量。

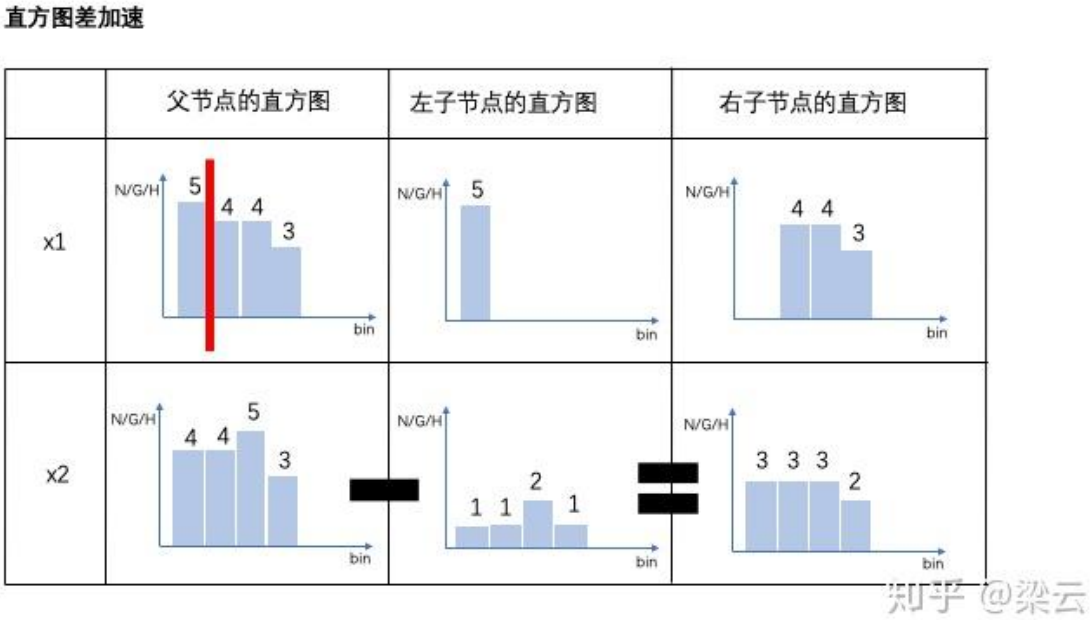


i	1	2	3	4	5	6	7	8
x_i	0.1	2.1	2.5	3.0	3.0	4.0	4.5	5.0
g_i	0.01	0.03	0.06	0.05	0.04	0.7	0.6	0.07
h_i	0.2	0.04	0.05	0.02	0.08	0.02	0.03	0.03

知乎 @梁云

i	1	2	3	4	5	6	7	8
x_i	0.1	2.1	2.5	3.0	3.0	4.0	4.5	5.0
g_i	0.01	0.03	0.06	0.05	0.04	0.7	0.6	0.07
h_i	0.2	0.04	0.05	0.02	0.08	0.02	0.03	0.03

$bini$	bin1	bin2	bin3
N_i	3	3	2
G_i	0.1	0.79	0.67
H_i	0.29	0.12	0.06



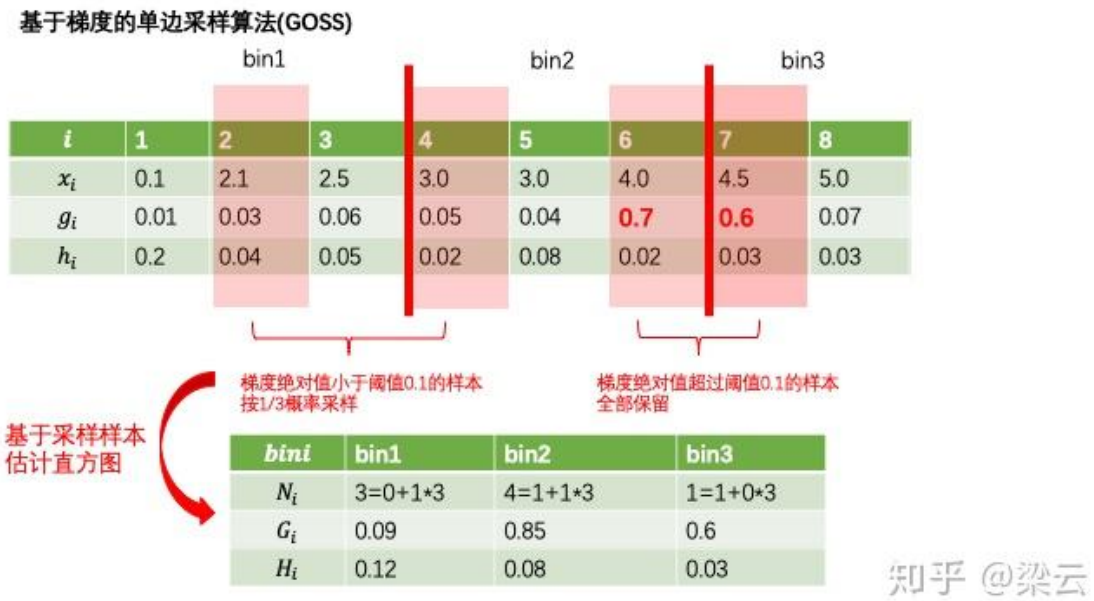
2.2 Goss Algorithm

GOSS 算法全称为 *Gradient-based One-Side Sampling*，即**基于梯度的单边采样算法。**

其**主要思想**是通过对样本采样的方法来减少计算目标函数增益时候的复杂度。但如果对全部样本进行随机采样，势必会对目标函数增益的计算精度造成较大的影响。

GOSS 算法的创新之处在于它只对梯度绝对值较小的样本按照一定比例进行采样，而保留了梯度绝对值较大的样本。这就是所谓的**单边采样**。由于目标函数增益主要来自于梯度

绝对值较大的样本，因此这种方法在计算性能和计算精度之间取得了很好的平衡。



i	1	2	3	4	5	6	7	8
x_i	0.1	2.1	2.5	3.0	3.0	4.0	4.5	5.0
g_i	0.01	0.03	0.06	0.05	0.04	**0.7**	**0.6**	0.07
h_i	0.2	0.04	0.05	0.02	0.08	0.02	0.03	0.03

$bini$	bin1	bin2	bin3
N_i	3=0+1*3	4=1+1*3	1=1+0*3
G_i	0.09	0.85	0.6
H_i	0.12	0.08	0.03

2.3 EFB Algorithm

EFB 算法全称是 $Exclusive\ Feature\ Bundling$，即**互斥特征绑定算法**。

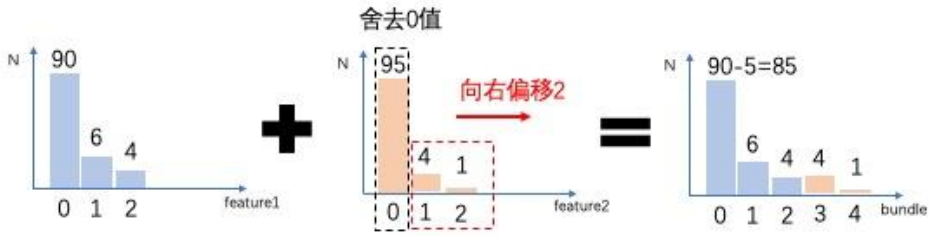
EFB 算法可以有效减少用于构建直方图的特征数量，从而降低计算复杂度，尤其是特征中包含大量稀疏特征的时候。

在许多应用场景下，数据集中会有大量的稀疏特征，这些稀疏特征大部分样本都取值为 0，只有少数样本取值非 0。通常可以认为这些稀疏特征是互斥的，即它们几乎不会同时取非零值。利用这种特性，可以通过对某些特征的取值重新编码，将多个这样互斥的特征捆绑成为一个新的特征。

有趣的是，对于类别特征，如果转换成 $onehot$ 编码，则这些 $onehot$ 编码后的多个特征相互之间是互斥的，从而可以被捆绑成为一个特征。

因此，对于指定为类别特征的特征，$LightGBM$ 可以直接将每个类别取值和一个 bin 关联，从而自动地处理它们。而无需预处理成 $onehot$ 编码多此一举。

互斥特征捆绑算法(EFB)



知乎 @梁云

3. $LightGBM$ Example

3.1 Code

```
import datetime
import numpy as np
import pandas as pd
import lightgbm as lgb
from sklearn import datasets
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score


def printlog(info):
    nowtime = datetime.datetime.now().strftime('%Y-%m-%d %H:%M:%S')
    print("\n"+"==========*8 + "%s"%nowtime)
    print(info+'...\n\n')

#==============================================================================
# 一，读取数据
#==============================================================================
printlog("step1: reading data ...")

# 读取 dftrain, dftest
breast = datasets.load_breast_cancer()
df = pd.DataFrame(breast.data, columns = [x.replace(' ', '_') for x in
breast.feature_names])
df['label'] = breast.target
```

```
df['mean_radius'] = df['mean_radius'].apply(lambda x: int(x))
df['mean_texture'] = df['mean_texture'].apply(lambda x: int(x))
dftrain, dftest = train_test_split(df)

categorical_features = ['mean_radius', 'mean_texture']
lgb_train = lgb.Dataset(
                                                dftrain.drop(['label'], axis
= 1),
                                                label=dftrain['label'],
                                                categorical_feature =
categorical_features
                                                )

lgb_valid = lgb.Dataset(
                                                dftest.drop(['label'],axis =
1),
                                                label=dftest['label'],
                        categorical_feature = categorical_features,
                        reference=lgb_train
                        )

#==============================================================================
# 二，设置参数
#==============================================================================
printlog("step2: setting parameters ...")

boost_round = 50
early_stop_rounds = 10

params = {
    'boosting_type': 'gbdt',
    'objective': 'binary',
    'metric': ['auc'],
    'num_leaves': 31,
    'learning_rate': 0.05,
    'feature_fraction': 0.9,
    'bagging_fraction': 0.8,
    'bagging_freq': 5,
    'verbose': 0
}

#==============================================================================
# 三，训练模型
#==============================================================================
printlog("step3: training model ...")
```

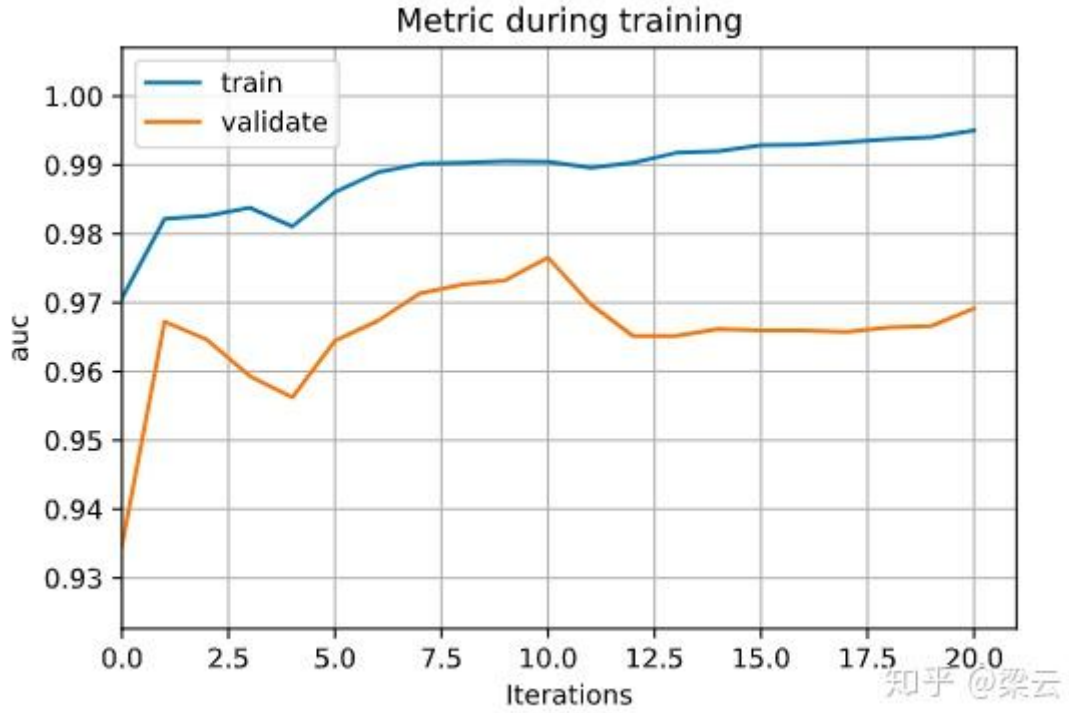
```
results = {}
gbm = lgb.train(params,
                lgb_train,
                num_boost_round=boost_round,
                valid_sets=(lgb_valid, lgb_train),
                valid_names=('validate', 'train'),
                early_stopping_rounds=early_stop_rounds,
                evals_result=results
                )

#==============================================================================
# 四，评估模型
#==============================================================================
printlog("step4: evaluating model ...")

y_pred_train = gbm.predict(
                                                    dftrain.drop('label',
axis=1),

num_iteration=gbm.best_iteration
                                                    )
y_pred_test = gbm.predict(
                                                   dftest.drop('label',
axis=1),

num_iteration=gbm.best_iteration
                                                   )

print('train accuracy: {:.5} '.format(accuracy_score(dftrain['label'],
y_pred_train > 0.5)))
print('valid accuracy: {:.5} \n'.format(accuracy_score(dftest['label'],
y_pred_test > 0.5)))

lgb.plot_metric(results)
lgb.plot_importance(gbm, importance_type = "gain")

#==============================================================================
# 五，保存模型
#==============================================================================
printlog("step5: saving model ...")


model_dir = "data/gbm.model"
print("model_dir: %s"%model_dir)
gbm.save_model("data/gbm.model")
printlog("task end ...")
```

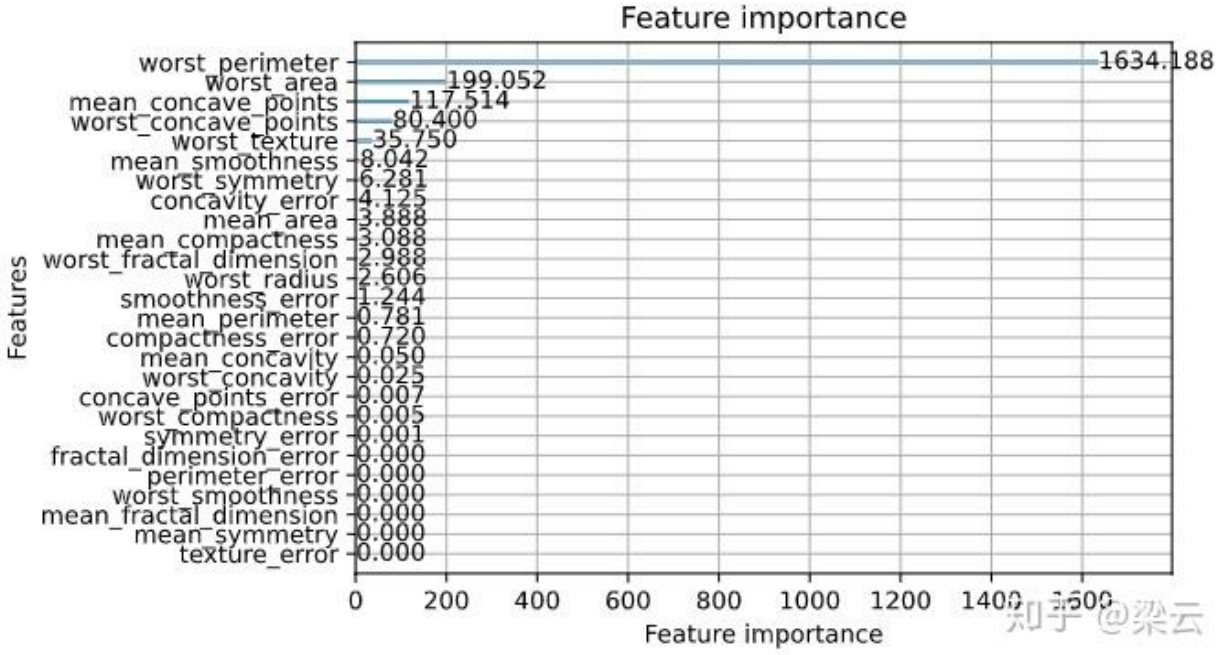
```
lgb.plot_metric(results,metric = "auc")

lgb.plot_importance(gbm,importance_type = "gain")
```

3.2 Output

```
==============================================================================
====2020-11-13 22:11:49
step1: reading data......



==============================================================================
====2020-11-13 22:11:49
step2: setting parameters......



==============================================================================
====2020-11-13 22:11:49
step3: training model......


[1]     train's auc: 0.98171    validate's auc: 0.972211
Training until validation scores don't improve for 10 rounds
[2]     train's auc: 0.988067   validate's auc: 0.984306
[3]     train's auc: 0.98885    validate's auc: 0.986705
[4]     train's auc: 0.987526   validate's auc: 0.989504
[5]     train's auc: 0.988706   validate's auc: 0.988904
[6]     train's auc: 0.989837   validate's auc: 0.990404
[7]     train's auc: 0.989524   validate's auc: 0.990404
[8]     train's auc: 0.990656   validate's auc: 0.990804
[9]     train's auc: 0.990993   validate's auc: 0.991204
[10]    train's auc: 0.991282   validate's auc: 0.991403
[11]    train's auc: 0.992438   validate's auc: 0.991903
[12]    train's auc: 0.992703   validate's auc: 0.991303
[13]    train's auc: 0.993016   validate's auc: 0.990304
[14]    train's auc: 0.993426   validate's auc: 0.990104
[15]    train's auc: 0.993715   validate's auc: 0.990904
[16]    train's auc: 0.993955   validate's auc: 0.991204
[17]    train's auc: 0.993787   validate's auc: 0.990804
[18]    train's auc: 0.99398    validate's auc: 0.991204
[19]    train's auc: 0.993859   validate's auc: 0.991403
[20]    train's auc: 0.993931   validate's auc: 0.991403
[21]    train's auc: 0.993955   validate's auc: 0.992203
[22]    train's auc: 0.994293   validate's auc: 0.991803
[23]    train's auc: 0.994533   validate's auc: 0.991803
```

```
[24]	training's auc: 0.994606	validation's auc: 0.992403
[25]	training's auc: 0.994678	validation's auc: 0.993003
[26]	training's auc: 0.995015	validation's auc: 0.993203
[27]	training's auc: 0.995376	validation's auc: 0.993802
[28]	training's auc: 0.995521	validation's auc: 0.994202
[29]	training's auc: 0.995738	validation's auc: 0.994402
[30]	training's auc: 0.996002	validation's auc: 0.995202
[31]	training's auc: 0.996171	validation's auc: 0.995202
[32]	training's auc: 0.996267	validation's auc: 0.995802
[33]	training's auc: 0.996604	validation's auc: 0.995802
[34]	training's auc: 0.996604	validation's auc: 0.996002
[35]	training's auc: 0.996629	validation's auc: 0.996202
[36]	training's auc: 0.996918	validation's auc: 0.996401
[37]	training's auc: 0.997255	validation's auc: 0.996601
[38]	training's auc: 0.997279	validation's auc: 0.996202
[39]	training's auc: 0.997495	validation's auc: 0.996202
[40]	training's auc: 0.997784	validation's auc: 0.997001
[41]	training's auc: 0.997784	validation's auc: 0.997001
[42]	training's auc: 0.997712	validation's auc: 0.997401
[43]	training's auc: 0.997809	validation's auc: 0.997401
[44]	training's auc: 0.997953	validation's auc: 0.997001
[45]	training's auc: 0.998001	validation's auc: 0.997001
[46]	training's auc: 0.998098	validation's auc: 0.997001
[47]	training's auc: 0.998146	validation's auc: 0.997001
[48]	training's auc: 0.998194	validation's auc: 0.996801
[49]	training's auc: 0.998266	validation's auc: 0.997001
[50]	training's auc: 0.998314	validation's auc: 0.996801
Did not meet early stopping. Best iteration is:
[50]	training's auc: 0.998314	validation's auc: 0.996801

==========================================================================
====2020-11-13 22:11:50
step4: evaluating model ......


train accuracy: 0.98122
valid accuracy: 0.95804


==========================================================================
====2020-11-13 22:11:50
step5: saving model ......


model_dir: data/gbm.model

==========================================================================
====2020-11-13 22:11:50
task end......
```


Metric during training
train
validate
auc
1.00
0.99
0.98
0.97
0.96
0.95
0.94
0.93
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
Iterations
知乎 @梁云


Feature importance
worst_perimeter 1634.188
worst_area 199.052
mean_concave_points 117.514
worst_concave_points 80.400
worst_texture 35.750
mean_smoothness 8.042
worst_symmetry 6.281
concavity_error 4.125
mean_area 3.888
mean_compactness 3.088
worst_fractal_dimension 2.988
worst_radius 2.606
smoothness_error 1.244
mean_perimeter 0.781
compactness_error 0.720
mean_concavity 0.050
worst_concavity 0.025
concave_points_error 0.007
worst_compactness 0.005
symmetry_error 0.001
fractal_dimension_error 0.000
perimeter_error 0.000
worst_smoothness 0.000
mean_fractal_dimension 0.000
mean_symmetry 0.000
texture_error 0.000
Features
0
200
400
600
800
1000
1200
1400
1600
Feature importance
知乎 @梁云